

Startup Ecosystem

Analytics Platform

Team Members

Srijan C Vachadmath	USN 1BM23CD060
Manohara Salmani	USN 1BM23CD035
Shreejth Shah	USN 1BM23CD057

Problem Statement

Relational DB Bottleneck

- ▶ Multi-hop lookups (e.g., co-investments) require complex recursive SQL JOINS.
- ▶ Query latencies degrade exponentially under deep traversals.

Concurrency & Oversubscription

- ▶ Concurrent funding calls read stale state, causing oversubscribed rounds.
- ▶ Lacks a thread-safe distributed lock manager.

Unverified Data Lifecycles

- ▶ Founders self-report accomplishments without third-party validation.
- ▶ No audit trail for investor due diligence.

Blocking API Execution

- ▶ Matchmaking queries block synchronous request loops.
- ▶ Real-time leaderboards and analytics cause UI lag under load.

Project Vision & Solution Overview

Platform Objective

High-performance startup analytics platform mapping founders, startups, and investors.

- ▶ **Graph Topology:** Native property graph stores nodes/edges directly.
- ▶ **Sub-15ms Matching:** Hybrid ranking computed in DB memory.

Multi-Role Portals

- ▶ **Startups:** Manage asks, post achievements, accept interest.
- ▶ **Investors:** Set ticket bounds, check matches, transfer funds.
- ▶ **Analysts:** Verify milestones, top up wallets, inspect paths.

Unified Value Proposition

Async FastAPI + Neo4j Graph + Redis cache & mutex = Scalable, secure startup directory.

Why Neo4j? — Graph Database Choice

Metric	Neo4j 5.13	Traditional SQL
Traversals	Index-free adjacency	Recursive JOINS
Edge Props Flexibility	Native properties Schemaless labels	Assoc. entities Rigid DDL migrations
Path query	Bolt (<2ms)	Multi-query loops

Connection Integrity

FastAPI lifespan configures persistent connection pool to `bolt://neo4j:7687` via Neo4j python driver, using auto-commit and transactional queries.

Use Case Rationale

- ▶ **Natural Structure:** Startup ecosystem is inherently relational. Connecting founders, startups, and investors via physical links.
- ▶ **Traversal Speed:** Evaluates path matching queries recursively in microseconds.

Why Redis? — High-Performance Caching

Feature	With Redis	Without Redis
Feed queries	Cached hit (<2ms)	DB query scan (50ms)
Match lists	TTL-based lookup	On-the-fly math loop
Round locks	Atomic lease (SET NX)	DB lock contention
Leaderboards	ZSET sorted list	Aggregated group-by

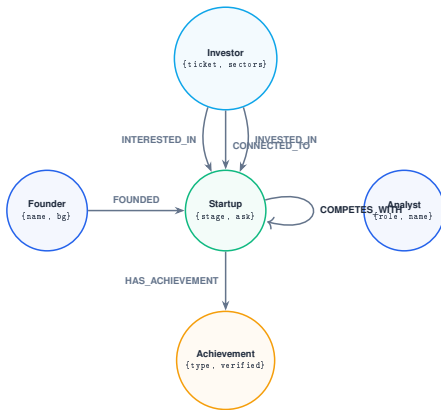
Revocation Control

JWT token IDs (JTI) blacklisted in Redis on logout for remaining token lifetime to enforce instant backend access revocation.

Caching Strategy

- ▶ **MD5 Hash Keys:** Keyed by query params, e.g. `feed:filtered:{md5}`.
- ▶ **Write Invalidation:** Mutations to profiles trigger immediate key deletes.
- ▶ **Atomic Lock Lease:** Prevents race conditions during funds transfer.

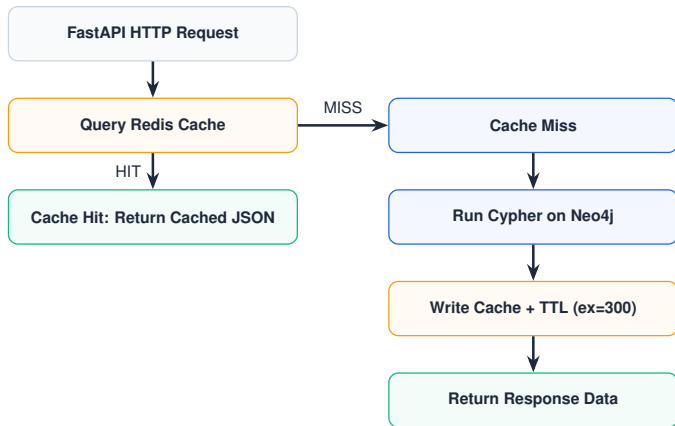
Neo4j Schema & Graph Design



Entity Constraints

- ▶ Uniqueness constraint on `Startup.id`.
- ▶ Uniqueness constraint on `Investor.id`.
- ▶ Property indexes on `sector` and `stage` attributes.

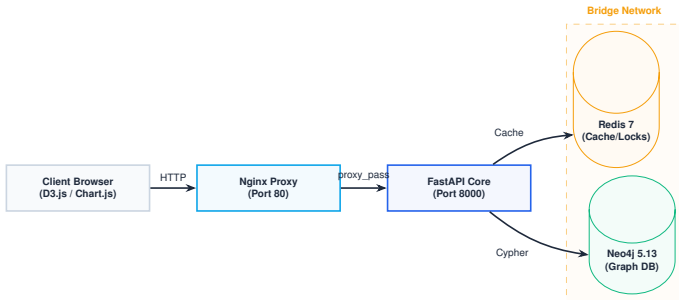
Redis Architecture & Data Flow



Storage Details

- ▶ **Hashes:**
`views:startup:{id}`
stores viewer metadata.
- ▶ **Sorted Sets:**
`leaderboard:investors`
rankings.
- ▶ **Strings:** JSON cached payload with explicit expiration.

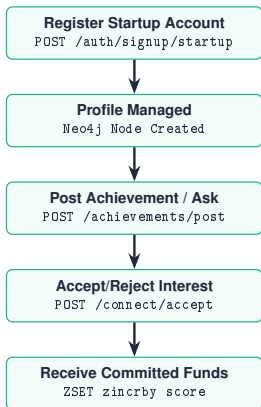
System Architecture Overview



Gateway Security

- ▶ Nginx exposes Port 80 and reverse proxies downstream.
- ▶ Restricts DB Bolt/Redis ports to private virtual bridge.

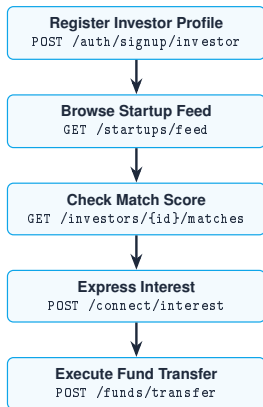
Startup User Journey



Key Workflow States

- ▶ **Unverified posts:** Post achievements which go into the queue.
- ▶ **Match feed:** Retrieve matched investors based on scoring.

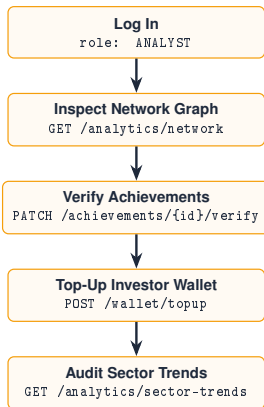
Investor User Journey



Financial Pipeline

- ▶ **Initial Capital:** Given \$500,000 wallet balance on signup.
- ▶ **Validation:** Asks cannot exceed available balance.

Analyst User Journey



Analytics Tools

- ▶ **Global view:** Displays co-investments and connections across the entire system.
- ▶ **Authority Actions:** Verifies startup achievements to update matches.

DevOps & Infrastructure Stack

Code & Build Automation

- ▶ **Git / GitHub:** Hosts source code and manifests.
- ▶ **Jenkins CI/CD:** 6-stage pipeline in `Jenkinsfile`.
- ▶ **GitHub Actions:** PR workflow validating code syntax.

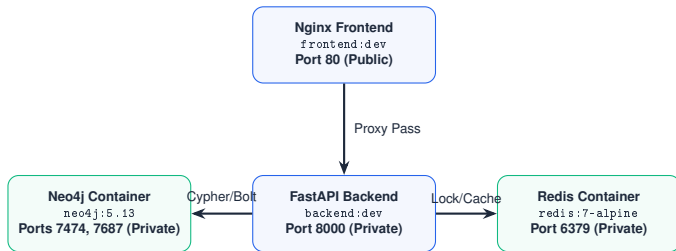
Serving & Storage

- ▶ **Nginx:** Proxy for static and dynamic endpoints.
- ▶ **Neo4j & Redis:** Property graph database and in-memory key-value cache.
- ▶ **Pytest Testing:** Boundary verification testing.

Virtualization & Runtime

- ▶ **Docker Containers:** Multi-stage builds for backend and frontend.
- ▶ **Docker Compose:** Configures isolated networks and volumes.

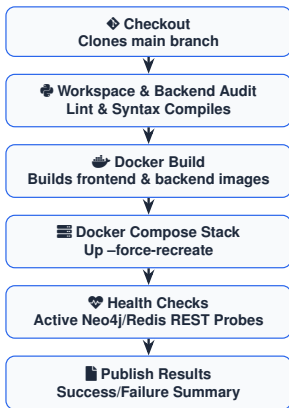
Containerization Strategy



Isolation Model

- ▶ Backend runs on a lightweight Debian-slim base image.
- ▶ Only Nginx binds to host Port 80, shielding API servers.
- ▶ Neo4j data folders mapped to persistent docker volumes.

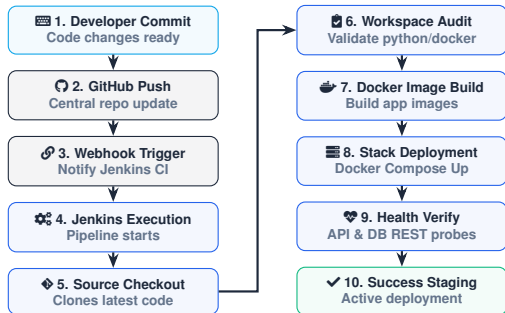
Jenkins Pipeline Implementation



Safety Gates

- ▶ Automatically cleans up orphan docker networks.
- ▶ Triggers rollback if backend port 8000 health pings fail.
- ▶ Seeds empty databases on deployment verify.

CI/CD Workflow – Startup Ecosystem Analytics Platform



CI/CD Pipeline Benefits

- ▶ **Automation:** Commits trigger build/test automatically.
- ▶ **Mitigation:** Linting catches bugs before deploy.
- ▶ **Containers:** Images guarantee environment consistency.
- ▶ **Reliability:** Health checks and rollback.

Pipeline Efficiency

Reduces manual overhead and ensures stable releases.

CI/CD Workflow – Overview & Benefits

Platform Overview

Automated CI/CD workflow using GitHub, Jenkins, Docker, and Docker Compose.

- ▶ **Continuous Integration:** Pushes trigger Jenkins webhook pings.
- ▶ **Automated Testing:** Workspace is linted, compiled, and audited.
- ▶ **Virtualization:** Services are containerized into Docker images.

Automated Pipeline

Validates, containerizes, and deploys changes automatically to improve reliability.

Benefits of the CI/CD Pipeline

- ✓ **Automated Builds:** Standardizes image creation.
- ✓ **Auto Deployment:** Seamless stack updates.
- ✓ **Early Detection:** Fails fast before production.
- ✓ **Consistency:** Avoids configuration drift.
- ✓ **Faster Releases:** Accelerates release iterations.
- ✓ **High Reliability:** Solidifies continuous delivery.
- ✓ **Seamless CI:** Integrates new features easily.
- ✓ **Fewer Errors:** Removes manual mistakes.

Supporting Services & Automation

Nginx Traffic Controller

- ▶ Handles static HTML file delivery directly, bypassing Python thread pools.
- ▶ Proxies dynamic `/health` and `auth` endpoints directly to FastAPI.
- ▶ Inject reverse proxy headers including `X-Real-IP`.

Environment Isolation

- ▶ Configuration managed through isolated `dotenv (.env)` bindings.
- ▶ Enables easy transitions between dev ports and staging settings.
- ▶ Dynamic startup hooks run database index statements automatically.

API Architecture & Route Schema

Core FastAPI Routers

- ▶ **Startups:** /startups/feed,
/startups/register,
/startups/{id}/matches
- ▶ **Investors:** /investors/register,
/investors/{id}/matches,
/investors/{id}/view/{sid}
- ▶ **Funds:** /funds/transfer,
/funds/history/{id}, /wallet/topup
- ▶ **Connections:** /connect/interest,
/connect/accept, /connect/reject
- ▶ **Achievements:** /achievements/post,
/achievements/{id}/verify

Validation Pipeline

- ▶ STRICT input checks via Pydantic schema models.
- ▶ StartupCreate & InvestorCreate reject invalid email strings or weak passwords (<6 chars) at the request layer.

Security Architecture & Access Controls

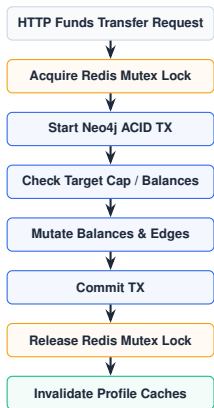
HMAC Token Signing

- ▶ Token payloads containing subject, name, role and unique JTI token ID are signed using HMAC-SHA256.
- ▶ Expired or malformed tokens raise an immediate 401 `Unauthorized` code.
- ▶ Revocations validated via Redis token blacklist checking.

Role Boundary Checks

- ▶ Custom python decorators enforce role parameters per route.
- ▶ Parametrized Cypher query syntax prevents NoSQL injection attempts.
- ▶ Enforces secure password storage via salted bcrypt hashes.

Transaction Lock Lifecycle



Protection Mechanics

- ▶ Mutex lock uses a 10s auto-expiry.
- ▶ Rejects transactions if startup funding limit is exceeded.
- ▶ ZSET and cache invalidation run on success.

Pytest Testing & Verification

Unit & Mock Framework

- ▶ Mock drivers simulate Neo4j endpoints and Redis pings in memory.
- ▶ Tests execute in isolation using GitHub Actions runner.
- ▶ Ensures database client failsafe boot modes work.

Boundary Conditions Validated

- ▶ Validates token signature tampering and expiry rejections.
- ▶ Confirms oversubscription triggers rollback.
- ▶ Validates RBAC rules for unauthorized route attempts (403 Forbidden).

Performance Optimization & Scalability

Caching Layer Strategies

- ▶ Cache key naming incorporates MD5 parameters to avoid collision.
- ▶ Read queries bypass database for 120s feed TTL.
- ▶ Profile updates execute clear cache commands asynchronously.

Graph Performance Metrics

- ▶ In-memory index scans on ID nodes reduce lookups to microseconds.
- ▶ Co-investment paths are evaluated with Cypher paths limited to 2 hops.
- ▶ Uses Redis sorted sets for zero-latency dashboard leaderboards.

Future Roadmap & Architecture Conclusion

Technical Growth Path

- ▶ Package stack configurations as Kubernetes Helm Charts.
- ▶ Apply Graph Data Science (GDS) algorithms for recommendations.
- ▶ Set up replica Redis caches for geographically distributed reads.

Architecture Summary

- ▶ **FastAPI Async Core:** High-concurrency throughput.
- ▶ **Neo4j Graph:** Native multi-hop relationship modeling.
- ▶ **Redis Mutex/Cache:** Prevents concurrency race conditions.